Static website generator

Calepin can build a Typst document directory as a static website. This is the same workflow used to render this documentation site: source `.typ` files live in `docs/`, and the generated `.html`, `.pdf`, and `sitemap.xml` files are written back into `docs/` for GitHub Pages.

New website

Use this command to scaffold a minimal site:

```
calepin new website
```

This creates:

- `docs/calepin.toml`: the site configuration file, where you set the title, base URL, navigation, theme, and output options.
- `docs/index.typ`: the home page source file, which builds to `index.html`.
- `docs/404.typ`: the not-found page source file, used by static hosts such as GitHub Pages for missing routes.

Build

Compile a website directory with the same `compile` command used for a single Typst document:

```
calepin compile docs/
```

This compiles the website in place: source files are read from `docs/`, and generated outputs are written back into `docs/`.

To compile the same source directory somewhere else, pass an output directory as the second path:

```
calepin compile docs/ public/
```

When the input path is a directory, *Calepin* looks for `calepin.toml` at the root of that directory. An explicit `--config <path>` supersedes the automatic lookup; if no config is found either way, the build fails.

By default, website pages render to HTML. Configure PDF output and page selection in Site configuration.

Serve

Serve the built site locally with:

```
calepin serve docs
```

This is useful for a quick preview after `calepin compile`. By default, *Calepin* uses `127.0.0.1` and the first available port from 8000.

Use `--host` and `--port` when you need a specific address:

```
calepin serve docs --host 127.0.0.1 --port 8001
```

Watch

During development, watch the website source directory:

```
calepin watch docs docs
```

As with `compile`, the first positional argument is the source directory and the optional second positional argument is the output directory. The initial watch build renders the whole site. After that, existing `.typ` page edits go through a fast xxh3 fingerprint lookup:

- If the changed page hash is new, *Calepin* rebuilds only that page.
- If the file was touched but the hash is unchanged, *Calepin* skips the rebuild.
- If navigation or page metadata changed after an incremental rebuild, *Calepin* falls back to a full rebuild.

Calepin also falls back to a full rebuild for changes that can affect more than one page:

- `calepin.toml`
- theme files
- assets
- new `.typ` pages
- removed `.typ` pages
- renamed `.typ` pages
- unknown or non-page inputs

Add `--serve` to run the local server while watching; it uses the same `--host` and `--port` options as `calepin serve`.

```
calepin watch docs docs --serve
```

Roadmap

Planned website features include:

- dependency-aware rebuilds for imported shared `.typ` files
- drafts
- taxonomies
- pagination
- search
- feeds
- robots.txt generation
- llms.txt generation
- link checking